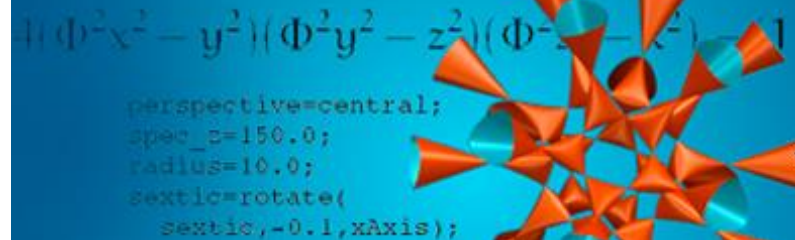


Theoretische Informatik: Äquivalenzsatz und Anwendungen

Prof. Dr. Elmar Tischhauser



Inhalt

- Bisher:
 - Reguläre Ausdrücke (REG, RE, RA) $\Rightarrow$ Syntaxdiagramm
 - NFA $\Rightarrow$ PA = DFA
 - ε -NFA $\Rightarrow$ DFA
- Jetzt:
 - REG $\Rightarrow$ (ε -)NFA
 - DFA $\Rightarrow$ REG
 - Äquivalenzsatz für reguläre Sprachen
 - flex

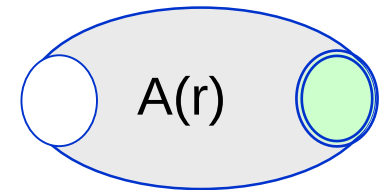
Reguläre Ausdrücke

- Gegeben: Regulärer Ausdruck r
- Problem:
 - Suche nach Teilstrings, die auf r passen.
- Idee:
 - Konstruiere einen Automaten A_r mit $L(A_r) = L(r)$.
 - String gefunden, wenn A_r einen Endzustand erreicht
- Strategie:
 1. Konstruiere einen Automaten mit ε -Transitionen (ε -NFA)
 2. Wandle diesen in deterministischen Automaten um
 3. Minimiere

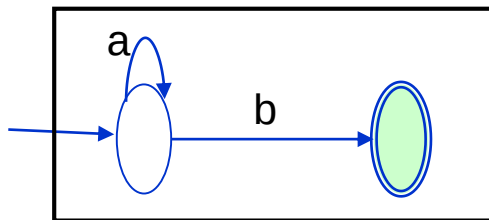
Reguläre Ausdrücke

Zu jedem **regulären Ausdruck** r gibt es einen endlichen Automaten $A(r)$ mit $L(r) = L(A(r))$.

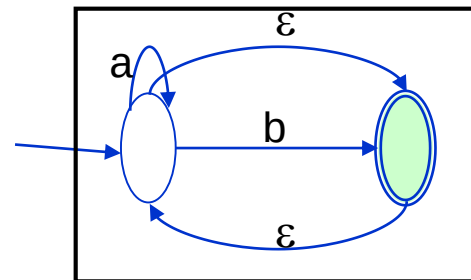
- Konstruktion induktiv über Aufbau von r



Ein bisschen Vorsicht ist notwendig:



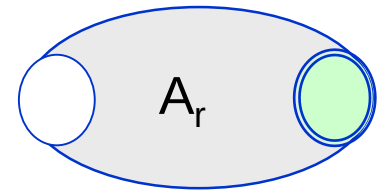
Automat für a^*b



Nicht Automat für $(a^*b)^*$

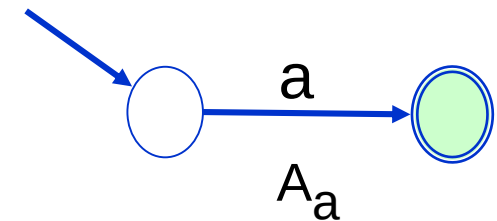
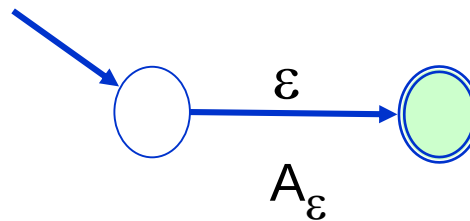
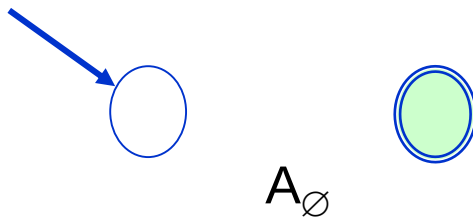
Induktive Konstruktion - Anfang

- Zu regulärem Ausdruck r konstruieren wir A_r mit folgenden Invarianten:
- A_r hat:
 1. genau einem Endzustand q_f
 2. keine Transition führt in den Anfangszustand
 3. keine Transition verlässt den Endzustand



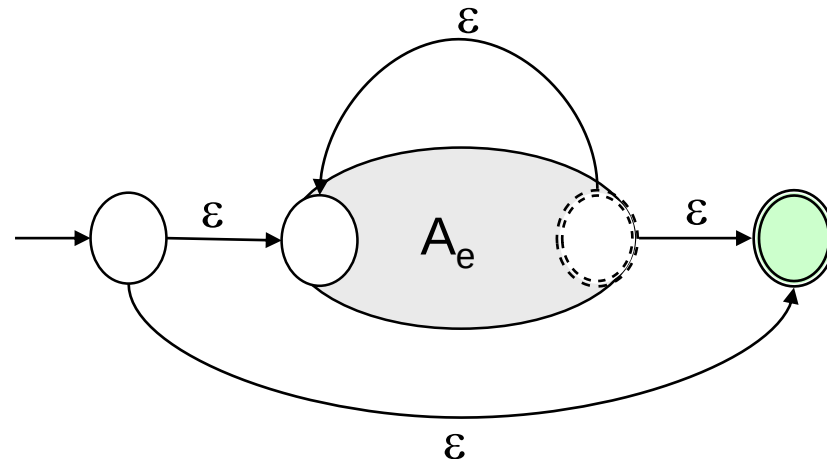
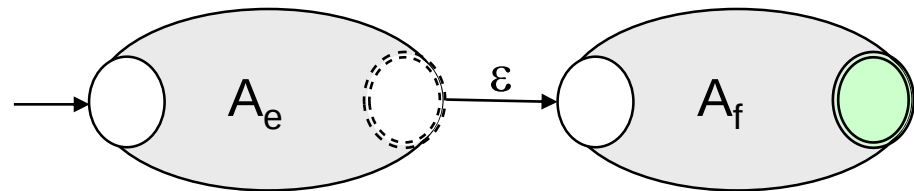
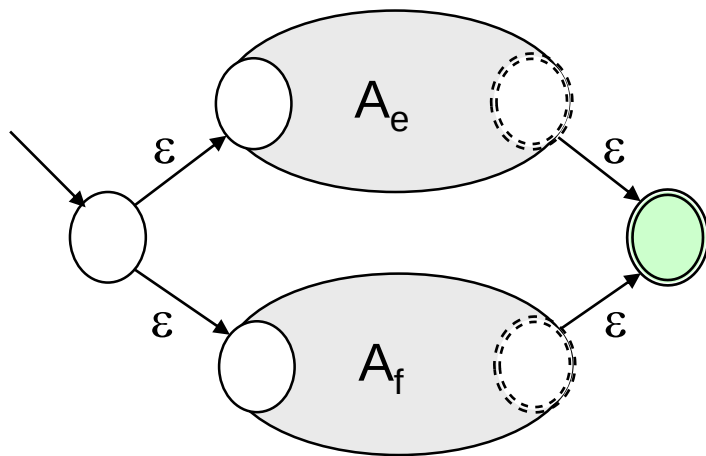
1. Konstante reguläre Ausdrücke

$\emptyset$, ε , a für jedes $a \in \Sigma$



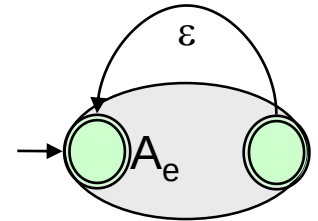
Induktive Konstruktion - Rekursion

2. Sind e und f reguläre Ausdrücke, dann auch
 $e+f$, ef , e^*

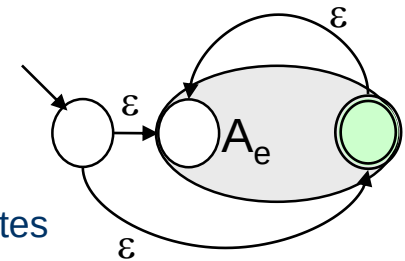


Aufgaben

- Der Student Hans Acker findet die Konstruktion des Automaten für e^* viel zu umständlich, weil zwei neue „sinnlose Zustände“ hinzukommen. Er schlägt vor
 - eine ε -Transition vom Endzustand in den Anfangszustand
 - den Anfangszustand auch zu einem Endzustand zu machen (Bild 1)

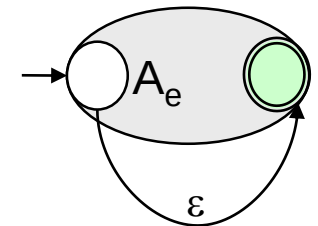


- Zeigen Sie H. Acker, dass seine Optimierung einen Bug hat. Finden Sie einen möglichst einfachen regulären Ausdruck e , so dass seine Konstruktion einen Automaten liefert, der mehr Worte erkennt, als nur $L(e^*)$.

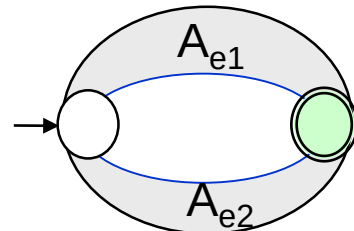


- Am nächsten Morgen hat H.Acker den Bug eingesehen und liefert sein verbessertes Programm. Es produziert den Automaten in Bild 2. Zeigen Sie ihm seinen Denkfehler.

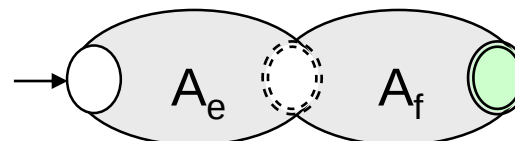
- $e?$ ist definiert als $e + \varepsilon$. Wie gewinnt man laut allgemeiner Konstruktion aus dem Automaten für e einen solchen für $e?$ H.Acker möchte optimieren und schlägt für $e?$ den Automaten in Bild 3 vor. Begründen Sie, dass diesmal seine Optimierung o.k. ist, aber nur, falls A_e die beiden Invarianten erfüllt



- O.K, aber die Vereinigung, den Automaten für $e_1 + e_2$, könnte man doch sicher einfacher haben, indem man die Anfangs- und Endzustände identifiziert wie in Bild 4. Begründen Sie, dass dies gut geht, wenn A_{e1} und A_{e2} beide Invarianten erfüllen.



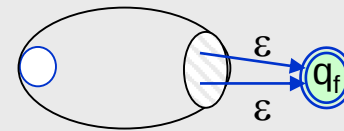
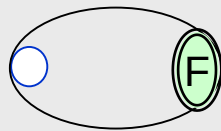
- Analoges gilt für die Konkatination



Automat (DFA) $\Rightarrow$ reg. Ausdruck

Zu jedem endlichen Automaten A gibt es einen regulären Ausdruck r_A mit $L(A) = L(r_A)$.

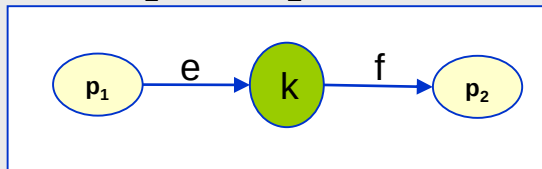
1. Ersetze Endzustände durch einen neuen Endzustand q_f



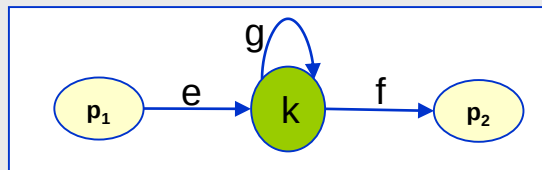
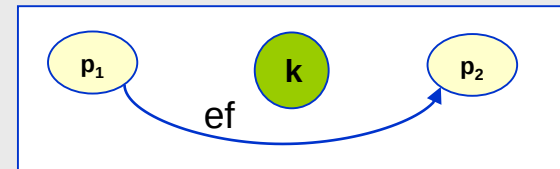
2. Elimination innerer Knoten ($k \notin \{q_0, q_f\}$)

Für je zwei Knoten $p_1 \neq k \neq p_2$

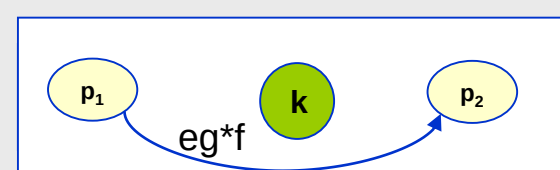
ersetze



durch



durch

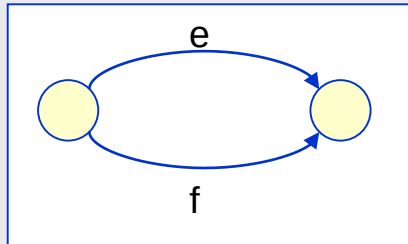


danach entferne k .

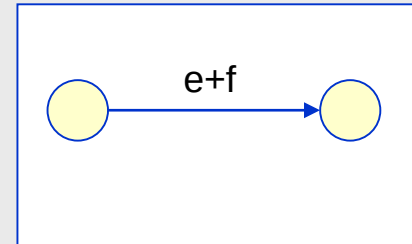
Automat (DFA) $\Rightarrow$ reg. Ausdruck

3. Elimination paralleler Kanten

Ersetze

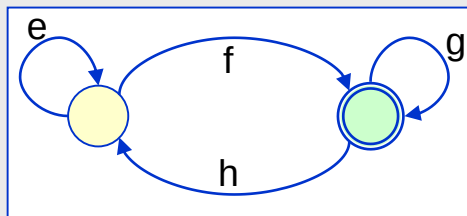


durch



4. Zum Schluss hat der Automat zwei Zustände. Ersetze ihn durch reg. Ausdruck

Ersetze

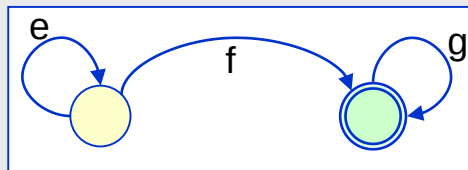


durch

$e^*f(g + he^*f)^*$

bzw.

etc.

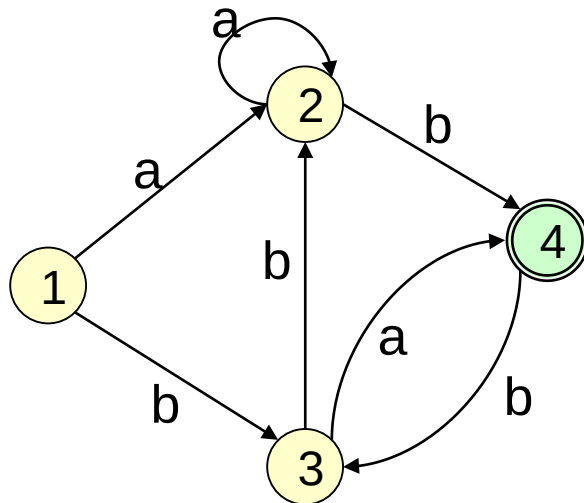


durch

e^*fg^*

Beispiel

- Konstruiere regulären Ausdruck für den Automaten



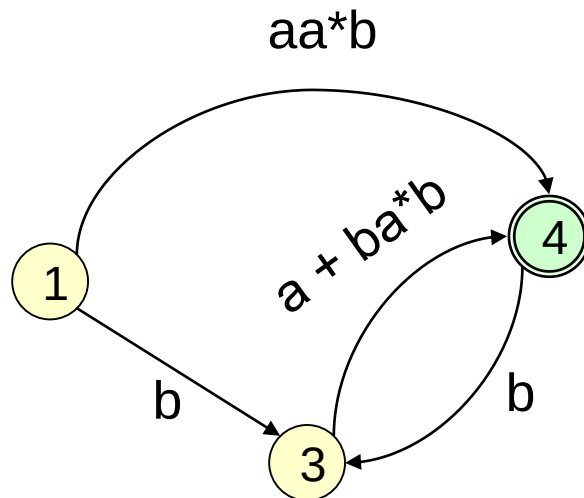
Nur ein Endzustand – o.k.

Eliminiere 2:

- Es gibt zwei Läufe über 2:
 - 1 aa^*b 4 und 3 ba^*b 4

Beispiel

- Konstruiere regulären Ausdruck für den Automaten



Nur ein Endzustand – o.k.

Eliminiere 2:

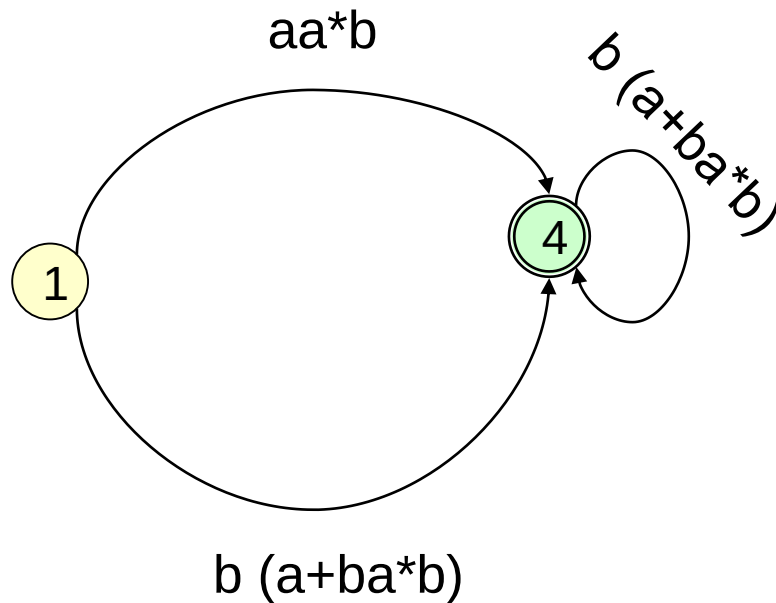
- Es gibt zwei Läufe über 2:
 - 1 aa^*b 4 und 3 ba^*b 4

Eliminiere 3:

- Es gibt zwei Läufe über 3:
 - 1 $b(a+ba^*b)$ 4 und
 - 4 $b(a+ba^*b)$ 4

Beispiel

- Konstruiere regulären Ausdruck für den Automaten



Nur ein Endzustand – o.k.

Eliminiere 2:

- Es gibt zwei Läufe über 2:
 - 1 aa^*b 4 und 3 ba^*b 4

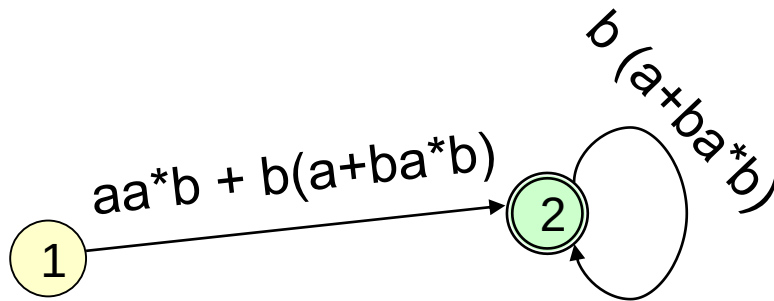
Eliminiere 3:

- Es gibt zwei Läufe über 3:
 - 1 $b(a+ba^*b)$ 4 und
 - 4 $b(a+ba^*b)$ 4

Verschmelze Kanten

Beispiel

- Konstruiere regulären Ausdruck für den Automaten



Nur ein Endzustand – o.k.

Eliminiere 2:

- Es gibt zwei Läufe über 2:
 - 1 aa^*b 4 und 3 ba^*b 4

Eliminiere 3:

- Es gibt zwei Läufe über 3:
 - 1 $b(a+ba^*b)$ 4 und
 - 4 $b(a+ba^*b)$ 4

Ergebnis:

$$(aa^*b + b(a+ba^*b))(b(a+ba^*b))^*$$

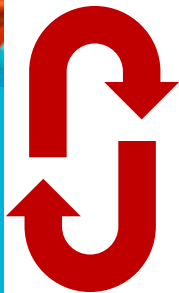
Verschmelze Kanten

Äquivalenzsatz für reguläre Sprachen

Äquivalenzsatz für reguläre Sprachen

- Reguläre Ausdrücke (RE),
- deterministische endliche Automaten (DFA),
- nichtdeterministische endliche Automaten (NFA),
- Automaten mit ε -Transitionen (ε -NFA)

erkennen/spezifizieren die gleichen regulären Sprachen.



- Zu jedem regulären Ausdruck e gibt es einen ε -NFA A_e , so dass $L(e)=L(A_e)$.
- Zu jedem ε -NFA A_e gibt es einen DFA PA mit $L(A_e) = L(PA)$
- Zu jedem DFA A_d gibt es einen regulären Ausdruck e_A mit $L(A_d) = L(e_A)$

Folgerungen

- Sind L , L_1 und L_2 reguläre Sprachen, dann sind auch regulär:

- $L_1 \cap L_2$ - Produktautomat
- $\Sigma^* - L$ - Komplementautomat
- $L_1 - L_2$ - $L_1 \cap (\Sigma^* - L_2)$

- Dennoch:

$\Sigma^* - L$ nicht monoton,
folglich nicht global mit regulären
Operatoren ausdrückbar

- in jedem Einzelfall neue Konstruktion erforderlich

Entscheidbare Probleme

- Gegeben reguläre Sprachen L, L_1, L_2
 - als reguläre Ausdrücke (RE)
 - als DFAs
 - als NFAs
- Aus dem Äquivalenzsatz folgt, dass alle möglichen Fragestellungen **entscheidbar** sind, z.B.:
 - Gegeben $w \in \Sigma^*$, ist $w \in L$?
 - Wortproblem
 - Ist $L_1 = L_2$?
 - Äquivalenzproblem
 - Ist $L = \emptyset$?
 - Leerheitsproblem

In der Praxis

1. Spezifiziere mit regulärem Ausdruck (RE)
 - Einfach zu bestimmen
 - Textuell repräsentierbar
2. Erzeuge äquivalenten ε -NFA
 - OK zur Spezifikation
 - Keine adäquate textuelle Darstellung
 - Bindeglied zwischen RE und DFA
3. Wandle um in DFA
 - Schlecht für Spezifizierung
 - Einfach zu implementieren
4. Minimiere
 - Entferne nicht erreichbare Zustände
 - Verschmelze äquivalente Zustände
5. Implementiere DFA tabellengesteuert
 - Sehr effizient



Dafür gibt es
Werkzeuge
z.B.: **flex**

Professionelles Werkzeug: flex

- flex ist ein Scanner-Generator
 - Aus regulärer Definition `r` erzeugt flex ein C-Programm, das einen Scanner für `r` implementiert
- Original UNIX-tool: `lex`
 - flex ist Open Source –Alternative
 - gleiche Input-Sprache
 - gleiches Verhalten
- Varianten für fast alle anderen Sprachen erhältlich
 - Java, Pascal, ML, Haskell, Prolog, ...

flex-Inputdatei

- drei Abschnitte, getrennt durch %%
 - Definitionen von token und Buchstabenklassen
 - %%
 - reguläre Ausdrücke mit Aktionen
 - %%
 - optional: main()

Beispieldatei für flex

```
letter  [a-zA-Z]
digit   [0-9]
%%
[ \t]+      ;
"+"        { return (PLUS)  }
"_"        { return (MINUS)  }
"*"        { return (TIMES)  }
"\"        { return (QUOT)   }
{digit}+    { sscanf(yytext,"%d",&yyval);
              return (NUM) }
{letter}({letter}|{digit})* { return (ID) }
%%
```

flex-Strategie

- Zu jedem regulären Ausdruck gehört eine **Aktion**
 - ausgeführt, sobald der Ausdruck erkannt wurde
 - falls Scanner front-end eines Compilers:
- Tokenübergabe: `return(Token);`
- beliebige zusätzliche C-Anweisungen möglich
- Aus Input wird **längstmöglicher Teilstring** abgeschnitten, der auf eines der Muster passt.

Bsp.: `dosen_Umfang := 3.14;`



`dosen_Umfang`

`:=`

`3.14`

`;`

`do`

`se`

`n_Umfang:`

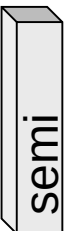
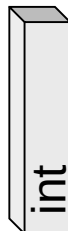
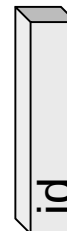
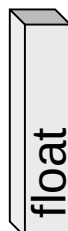
`=`

`3`

`.`

`14`

`;`



flex auch sonst nützlich

- flex ist für viele Zwecke nützliches Werkzeug
 - Beispiel: Schreibe Programm um
 - alle Großbuchstaben in Kleinbuchstaben zu verwandeln
 - Leerzeichen am Zeilenende zu entfernen
 - mehrfache Leerzeichen durch eins zu ersetzen
- Input-Datei „myFile.l“ mit Inhalt

```
# Dies ist die Datei „myFile.lxi“
%%
[A-Z]  putchar(yytext[0]+'a'-'A');
[ ]+$  ;
[ ]+   putchar(' ');
%%
```

Interne Variable **yytext[]** enthält
stets den zuletzt erkannten String

Console:

```
> flex myFile.lxi
> cc lex.yy.c -ll
> mv a.out myProg
```

-ll linkt mit der **lex-library**

On the fly scanning

- Scannen ohne Konstruktion eines Automaten

- r regulärer Ausdruck und w Eingabewort.

Ist $w \in L(r)$?

- Idee:

$$\begin{aligned} a.u \in L(r) &\Leftrightarrow u \in L(r)_a \\ &\Leftrightarrow u \in L(\partial_a(r)) \end{aligned}$$

- also

$$w \in L(r) \Leftrightarrow \varepsilon \in L(\partial_w(r))$$

Differentiation regulärer Ausdrücke

- Sei $a \in \Sigma$ und r ein regulärer Ausdruck

$$L(\partial_a(r)) := \{v \in \Sigma^* \mid a.v \in L(r)\}$$

- $\partial_a(r)$ heißt Ableitung von r nach a

$$u \in L(\partial_a(r)) \Leftrightarrow a.u \in L(r)$$

- Einfache Differentiationsregeln:

- $\partial_a(\emptyset) = \emptyset$, $\partial_a(\varepsilon) = \emptyset$
- $\partial_a(a) = \varepsilon$, $\partial_a(b) = \emptyset$, falls $a \neq b \in \Sigma$
- $\partial_a(r_1 + r_2) = \partial_a r_1 + \partial_a r_2$
- $\partial_a(r^*) = (\partial_a r)^*$
- $\partial_a(r_1 r_2) = \text{if } (\varepsilon \notin L(r_1)) \text{ then } (\partial_a r_1) r_2$
else $(\partial_a r_1) r_2 + \partial_a r_2$

On the fly scanning

- Beginne mit dem Paar (w, r)
 - w restliches Eingabewort,
 - r regulärer Ausdruck
- In jedem Schritt wird w verkürzt und r entsprechend modifiziert:
 - $w = a_1 a_2 a_3 \dots a_n \in L(r) \Leftrightarrow a_2 a_3 \dots a_n \in L(\partial_{a_1}(r))$
 $\Leftrightarrow a_3 \dots a_n \in L(\partial_{a_2}(\partial_{a_1}(r)))$
 $\Leftrightarrow \dots$
 $\Leftrightarrow \varepsilon \in L(\partial_w(r))$
- Beispiel: $aa \in L(a(ab)^*) \Leftrightarrow a \in L((ab)^*) \Leftrightarrow \varepsilon \in L(b(ab)^*) \Leftrightarrow false$

Differentiation nach Worten

- Erweiterung der Differentiation auf Worte

- $\partial_\varepsilon(L) := L$
- $\partial_{a.v}(L) := \partial_v(\partial_a(L))$

- Für jedes Wort w gilt dann

$$w \in L \Leftrightarrow \varepsilon \in \partial_w(L)$$

- Beweis durch Induktion über Aufbau von w
- Die letzte Formel liefert ein Kriterium, um festzustellen, ob ein Wort w in der Sprache L ist
 - Wir müssen
 - $\partial_w(L)$ berechnen
 - feststellen, ob ε in der erhaltenen Sprache ist

Begründung der Regel

- Regel: $\partial_a(r_1 r_2) = \text{if } (\varepsilon \notin L(r_1)) \text{ then } (\partial_a r_1) r_2$
else $(\partial_a r_1) r_2 + \partial_a r_2$

- Beweis:

- $$u \in L(\partial_a(r_1 r_2)) \Leftrightarrow a.u \in L(r_1 r_2)$$

$$\Leftrightarrow \exists u_1, u_2: a.u = a.u_1 u_2, a.u_1 \in L(r_1), u_2 \in L(r_2)$$

$$\text{oder } (\varepsilon \in L(r_1), a.u \in L(r_2))$$

$$\Leftrightarrow \exists u_1, u_2: a.u = a.u_1 u_2, u_1 u_2 \in L((\partial_a r_1) r_2),$$

$$\text{oder } (\varepsilon \in L(r_1), u \in L(\partial_a r_2))$$

$$\Leftrightarrow u \in L((\partial_a r_1) r_2),$$

$$\text{oder } (\varepsilon \in L(r_1), u \in L(\partial_a r_2))$$

$$\Leftrightarrow \text{if } (\varepsilon \notin L(r_1)) \text{ then } u \in L((\partial_a r_1) r_2)$$

$$\text{else } u \in (L((\partial_a r_1) r_2) + L(\partial_a r_2))$$

$$\Leftrightarrow u \in \text{if } (\varepsilon \notin L(r_1)) \text{ then } L((\partial_a r_1) r_2)$$

$$\text{else } (L((\partial_a r_1) r_2) + L(\partial_a r_2))$$

Vereinfachung der Notation

Reguläre Ausdrücke

- Mit der Notation
 - $0 := \{ \}$, $1 := \{ \varepsilon \}$
 - $a := \{ a \}$ für jedes $a \in \Sigma$
- erhalten wir die Gleichungen
 - $0 + L = L = L + 0$
 - $L \cdot 0 = 0 = 0 \cdot L$
- L heißt **nullable**, falls $\varepsilon \in L$.
- Wir setzen
 - $v(L) := L \cap \{ \varepsilon \}$
= if($\varepsilon \in L$) then 1 else 0

Differentiation

- Statt $\partial_a(L)$ schreiben wir $a^{-1}L$ oder auch **$a^{-1}L$**
- Differentiationsregeln daher:
 - $a^{-1}0 = 0 = a^{-1}1$
 - $a^{-1}a = 1$
 - $a^{-1}b = 0$, falls $a \neq b \in \Sigma$
 - $a^{-1}(L_1 + L_2) = a^{-1}(L_1) + a^{-1}(L_2)$
 - $a^{-1}(L^*) = a^{-1}(L)L^*$
 - $a^{-1}(L_1 L_2) = a^{-1}(L_1)L_2 + v(L_1)a^{-1}(L_2)$
- Für Worte u, v, ε erhält man
 - $\varepsilon^{-1}(L) = L$
 - $(uv)^{-1}L = v^{-1}u^{-1}L$
- **Vorsicht:** $a^{-1}(uv) \neq (a^{-1}u)v$

Notation:

$0 := \{ \}$, $1 := \{ \varepsilon \}$

$a := \{a\}$ für jedes $a \in \Sigma$

Erkennung ohne Automaten

- Gegeben regulärer Ausdruck r und Wort w

- Frage: Ist $w \in L(r)$?

- Äquivalent: Prüfe ob

$$\varepsilon \in \partial_w(r), \text{ d.h. } v(w^{-1}(r)) = 1$$

- Beispiel: Ist $abba \in L(a(ba^*)^*)$?

$$\begin{aligned} - (abba)^{-1}(a(ba^*)^*) &= (bba)^{-1}a^{-1}(a(ba^*)^*) = (bba)^{-1} 1(ba^*)^* = (ba)^{-1} b^{-1}(ba^*)^* = \\ &= (ba)^{-1} b^{-1}(ba^*)(ba^*)^* = (ba)^{-1} 1a^*(ba^*)^* = a^{-1} b^{-1} (a^*(ba^*)^*) \\ &= a^{-1} ((b^{-1} a^*)(ba^*)^* + b^{-1} (ba^*)^*) = a^{-1} (0 + (b^{-1} (ba^*)^*)) \\ &= a^{-1} (b^{-1}(ba^*) (ba^*)^*) = a^{-1} (a^*(ba^*)^*) \\ &= a^{-1}(a^*)(ba^*)^* + a^{-1}(ba^*)^* = a^{-1}(a) a^*(ba^*)^* + a^{-1}(ba^*)^* \\ &= a^*(ba^*)^* + 0 = a^*(ba^*)^* \end{aligned}$$

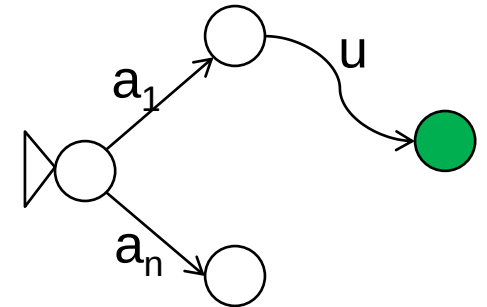
$$v(a^*(ba^*)^*) = 1, \text{ also } abba \in L(a(ba^*)^*)$$

$a^{-1}(a(ba^*)^*) \rightarrow$ Konkatenationsregel:
 $a^{-1}(L_1 L_2) = a^{-1}(L_1) L_2 + v(L_1) a^{-1}(L_2)$
 $L_1 = a, L_2 = (ba^*)^*$

$$\begin{aligned} a^{-1}(a(ba^*)^*) &= 1(ba^*)^* + v(a) a^{-1}((ba^*)^*) \\ &= 1(ba^*)^* + 0 a^{-1}(ba^*)(ba^*)^* \rightarrow \text{Konka...} \\ &= 1(ba^*)^* + 0 a^{-1}(b) a^* + v(b) a^{-1}(a^*)(ba^*)^* \\ &= 1(ba^*)^* + 0 0 a^* + 0 a^{-1}(a^*)(ba^*)^* \\ &= 1(ba^*)^* + 0 0 a^* + 0 a^{-1}(a) a^*(ba^*)^* \\ &= 1(ba^*)^* + 0 0 a^* + 0 1 a^*(ba^*)^* \\ &= 1(ba^*)^* + 0 0 a^* \\ &= 1(ba^*)^* \end{aligned}$$

Ableitung und Automaten

- Sei r ein regulärer Ausdruck und
 - $A(r) = (Q, \Sigma, \delta, q_0, F)$ ein Automat für $L(r)$



Dann gilt

- $A(a^{-1}r) = (Q, \Sigma, \delta, \delta(q_0, a), F)$, für $a \in \Sigma$, und
 - $A(w^{-1}r) = (Q, \Sigma, \delta, \delta^*(q_0, w), F)$, für $w \in \Sigma^*$.
- Sei $\Sigma = \{a_1, \dots, a_n\}$, dann gilt für jede Sprache $L \subseteq \Sigma^*$:
 - $L = a_1(a_1^{-1}L) + \dots + a_n(a_n^{-1}L) + v(L)$

Zusammenfassung

- Äquivalenz der Konzepte

- Regulärer Ausdruck $\Rightarrow$ NFA
- DFA $\Rightarrow$ regulärer Ausdruck
- DFA $\Leftrightarrow$ NFA voriges Kapitel

- flex

- Scanner Generator
- Moore Automat aus regulären Ausdrücken
- vielseitig einsetzbar

- On the fly-scanning

- Automaten-Konstruktion nicht notwendig
- Stattdessen Differentiation
 - $w \in L(r) \Leftrightarrow \varepsilon \in \hat{w}(r)$

- Berry-Sethi Algorithmus

- Aus r konstruiere direkt kleinen NFA
- praktisch gut einsetzbar
- Einfach implementierbar